GLYF Cathedral Android — Complete Source Code

Version: 0.7.2 φ⁷ **Date:** 2026-03-23 **Total Lines:** ~2,500 **Language:** Kotlin

---

Table of Contents

1. [Core Models](#core-models) 2. [Data Layer](#data-layer) 3. [UI Layer](#ui-layer) 4. [Cron System](#cron-system) 5. [Rosetta Bridge](#rosetta-bridge) 6. [Build Configuration](#build-configuration)

Core Models

`core/Models.kt` (349 lines)

```
package com.glyf.cathedral.core

import kotlin.math.abs
import kotlin.math.pow

/**
 * φ⁷ Geometric Constants
 */
object PhiConstants {
    const val PHI = 1.618033988749895
    const val PHI_INV = 0.6180339887498948
    const val PHI_2 = 2.618033988749895
    const val PHI_3 = 4.23606797749979
    const val PHI_4 = 6.854101966249685
    const val PHI_7 = 29.034441853748633
    const val GOLDEN_ANGLE_DEG = 137.50776405003785
    const val GOLDEN_ANGLE_RAD = 2.39996322972865332

    // Fibonacci sequence mod 89 for cron intervals
    val FIB_MOD_89 = listOf(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 % 89, 144 % 89, 233 % 89)

    /**
     * Compute φ^k with saturation
     */
    fun phiPow(k: Int): Double = when (k) {
        0 -> 1.0
        1 -> PHI
        2 -> PHI_2
        3 -> PHI_3
        4 -> PHI_4
        7 -> PHI_7
        -1 -> PHI_INV
        else -> PHI.pow(k)
    }
}

/**
 * Axial coordinates for hexagonal grid
 * Constraint: q + r + s = 0
 */
data class AxialCoord(
    val q: Int,
    val r: Int,
    val s: Int
) {
    init {
        require(q + r + s == 0) { "Axial coordinates must sum to zero: ($q, $r, $s)" }
    }

    companion object {
        val ORIGIN = AxialCoord(0, 0, 0)

        fun fromQr(q: Int, r: Int) = AxialCoord(q, r, -q - r)
    }

    /**
     * Hex distance from origin
     */
    fun distance(): Int = (abs(q) + abs(r) + abs(s)) / 2

    /**
     * Hex distance to another coordinate
     */
```

```
    fun distanceTo(other: AxialCoord): Int {
        val dq = q - other.q
        val dr = r - other.r
        val ds = s - other.s
        return (abs(dq) + abs(dr) + abs(ds)) / 2
    }

    /**
     * Convert to pixel coordinates (pointy-topped hex)
     */
    fun toPixel(size: Float): Pair {
        val x = size  (sqrt3  q + sqrt3 / 2 * r)
        val y = size  (3.0 / 2  r)
        return Pair(x.toFloat(), y.toFloat())
    }

    /**
     * Neighbors in hex grid (6 directions)
     */
    fun neighbors(): List = listOf(
        AxialCoord(q + 1, r, s - 1),
        AxialCoord(q + 1, r - 1, s),
        AxialCoord(q, r - 1, s + 1),
        AxialCoord(q - 1, r, s + 1),
        AxialCoord(q - 1, r + 1, s),
        AxialCoord(q, r + 1, s - 1)
    )

    private val sqrt3 = 1.7320508075688772
}

/**
 * Ternary spin states
 */
enum class TernarySpin(val value: Byte) {
    NEGATIVE(-1),
    ZERO(0),
    POSITIVE(1);

    companion object {
        fun fromValue(v: Byte): TernarySpin = when (v) {
            (-1).toByte() -> NEGATIVE
            0.toByte() -> ZERO
            1.toByte() -> POSITIVE
            else -> ZERO
        }
    }
}

/**
 * HexTile — persistence tile for φ⁷ lattice
 * Android adaptation (64-byte alignment not required)
 */
data class HexTile(
    val coord: AxialCoord,
    val spin: TernarySpin = TernarySpin.ZERO,
    val phiMag: Float = 0.0f,
    val chiralHash: Long = 0L,
    val timestamp: Long = System.nanoTime(),
    val evictionPriority: Byte = 128,
    val priorityTiebreaker: Byte = 0
) {
    companion object {
        const val MAX_PRIORITY: Byte = 255
        const val MIN_PRIORITY: Byte = 0
    }

    /**
     * Compute distance from origin
     */
    fun distance(): Int = coord.distance()

    /**
     * Scale phiMag by φ^k with saturation
     */
    fun scalePhi(k: Int): HexTile {
        val newMag = (phiMag * PhiConstants.phiPow(k)).toFloat()
        val saturated = when {
            newMag < 1.17549435e-38f -> 0.0f // f32 min normal
            newMag > PhiConstants.PHI_7.toFloat() -> PhiConstants.PHI_7.toFloat()
            else -> newMag
        }
        return copy(phiMag = saturated)
    }

    /**
```

```
     * Encode chiral hash from 3 spin populations
     */
    fun withChiralHash(spinNeg: Int, spinZero: Int, spinPos: Int): HexTile {
        val mask0 = (spinNeg and 0x1FFFFF).toLong()
        val mask1 = ((spinZero and 0x1FFFFF).toLong() shl 21)
        val mask2 = ((spinPos and 0x3FFFFF).toLong() shl 42)
        return copy(chiralHash = mask0 or mask1 or mask2)
    }

    /**
     * Decode chiral hash to 3 spin populations
     */
    fun decodeChiral(): Triple {
        val spinNeg = (chiralHash and 0x1FFFFF).toInt()
        val spinZero = ((chiralHash shr 21) and 0x1FFFFF).toInt()
        val spinPos = ((chiralHash shr 42) and 0x3FFFFF).toInt()
        return Triple(spinNeg, spinZero, spinPos)
    }

    /**
     * Update eviction priority based on distance
     */
    fun withPriority(maxRadius: Int): HexTile {
        val dist = distance()
        val maxR = maxRadius.coerceAtLeast(1)
        val priority = ((dist * 255) / maxR).coerceIn(0, 255).toByte()
        val tiebreaker = ((timestamp shr 32).toInt() xor dist).toByte()
        return copy(
            evictionPriority = priority,
            priorityTiebreaker = tiebreaker
        )
    }

    /**
     * Check if this tile should be evicted before other
     */
    fun shouldEvictBefore(other: HexTile): Boolean {
        return if (evictionPriority != other.evictionPriority) {
            evictionPriority > other.evictionPriority // Higher = evict first
        } else {
            priorityTiebreaker > other.priorityTiebreaker
        }
    }
}

/**
 * Cron tile for scheduled persistence
 */
data class CronTile(
    val intervalFib: Byte, // τ mod 89
    val lastTick: Long = 0, // Unix seconds
    val voltageThreshold: Float = PhiConstants.PHI_INV.toFloat(),
    val anchorCoord: AxialCoord
) {
    companion object {
        const val DEFAULT_INTERVAL: Byte = 13
    }

    /**
     * Get interval in seconds
     */
    fun intervalSeconds(): Long {
        val fib = PhiConstants.FIB_MOD_89.getOrElse(intervalFib.toInt()) { 13 }
        return fib * 60L
    }

    /**
     * Check if cron should fire
     */
    fun shouldFire(nowSecs: Long, currentVoltage: Float): Boolean {
        val elapsed = nowSecs - lastTick
        return elapsed >= intervalSeconds() && currentVoltage >= voltageThreshold
    }

    /**
     * Record a tick
     */
    fun tick(nowSecs: Long): CronTile = copy(lastTick = nowSecs)

    /**
     * Time until next fire (saturating)
     */
    fun timeUntilFire(nowSecs: Long): Long {
        val interval = intervalSeconds()
        val elapsed = nowSecs - lastTick
        return (interval - elapsed).coerceAtLeast(0)
```

```
    }
}


/**
 * Lattice state container
 */
data class LatticeState(
    val version: Int = 0x00000702,
    val hotTiles: List = emptyList(),
    val cronTiles: List = emptyList(),
    val explorerVoltage: Float = PhiConstants.PHI_7.toFloat(),
    val junctionThreshold: Byte = (-3).toByte(),
    val radialRadius: Short = 21,
    val cronFibBase: Byte = 13
) {
    companion object {
        const val MAX_HOT_TILES = 1024
        const val MAX_CRON_TILES = 64
    }

    val hotCount: Int get() = hotTiles.size
    val cronCount: Int get() = cronTiles.size
    val loadFactor: Float get() = hotCount.toFloat() / MAX_HOT_TILES

    /**
     * Insert tile, evicting if necessary
     */
    fun insertTile(tile: HexTile): Pair {
        return if (hotCount >= MAX_HOT_TILES) {
            // Evict outermost
            val sorted = hotTiles.sortedWith(compareByDescending { it.evictionPriority })
            val evicted = sorted.firstOrNull()
            val remaining = sorted.drop(1).take(MAX_HOT_TILES - 1)
            val newTiles = remaining + tile.withPriority(radialRadius.toInt())
            Pair(copy(hotTiles = newTiles), evicted)
        } else {
            val newTiles = hotTiles + tile.withPriority(radialRadius.toInt())
            Pair(copy(hotTiles = newTiles), null)
        }
    }

    /**
     * Find tile by coordinate
     */
    fun findTile(coord: AxialCoord): HexTile? = hotTiles.find { it.coord == coord }

    /**
     * Query tiles within radial distance
     */
    fun queryRadial(center: AxialCoord, radius: Int): List {
        return hotTiles.filter { it.coord.distanceTo(center) <= radius }
    }

    /**
     * Process cron ticks
     */
    fun processCrons(nowSecs: Long): Pair {
        var fired = 0
        val updatedCrons = cronTiles.map { cron ->
            if (cron.shouldFire(nowSecs, explorerVoltage)) {
                fired++
                cron.tick(nowSecs)
            } else {
                cron
            }
        }
        return Pair(copy(cronTiles = updatedCrons), fired)
    }

    /**
     * Scale explorer voltage by φ^k
     */
    fun scaleVoltage(k: Int): LatticeState {
        val newVoltage = (explorerVoltage * PhiConstants.phiPow(k)).toFloat()
        val saturated = when {
            newVoltage < 1.17549435e-38f -> 0.0f
            newVoltage > PhiConstants.PHI_7.toFloat() -> PhiConstants.PHI_7.toFloat()
            else -> newVoltage
        }
```

```
        return copy(explorerVoltage = saturated)
    }
}
```

Data Layer

`data/Entities.kt` (192 lines)

```
package com.glyf.cathedral.data

import androidx.room.*
import com.glyf.cathedral.core.*

/**
 * Room Entity for HexTile
 */
@Entity(tableName = "hex_tiles")
data class HexTileEntity(
    @PrimaryKey
    val id: String, // "q,r,s" format

    val q: Int,
    val r: Int,
    val s: Int,

    val spinValue: Byte,
    val phiMag: Float,
    val chiralHash: Long,
    val timestamp: Long,
    val evictionPriority: Byte,
    val priorityTiebreaker: Byte
) {
    companion object {
        fun fromTile(tile: HexTile): HexTileEntity {
            val coord = tile.coord
            return HexTileEntity(
                id = "${coord.q},${coord.r},${coord.s}",
                q = coord.q,
                r = coord.r,
                s = coord.s,
                spinValue = tile.spin.value,
                phiMag = tile.phiMag,
                chiralHash = tile.chiralHash,
                timestamp = tile.timestamp,
                evictionPriority = tile.evictionPriority,
                priorityTiebreaker = tile.priorityTiebreaker
            )
        }
    }

    fun toTile(): HexTile = HexTile(
        coord = AxialCoord(q, r, s),
        spin = TernarySpin.fromValue(spinValue),
        phiMag = phiMag,
        chiralHash = chiralHash,
        timestamp = timestamp,
        evictionPriority = evictionPriority,
        priorityTiebreaker = priorityTiebreaker
    )
}

/**
 * Room Entity for CronTile
 */
@Entity(tableName = "cron_tiles")
data class CronTileEntity(
    @PrimaryKey
    val id: String, // "q,r,s" anchor format

    val intervalFib: Byte,
    val lastTick: Long,
    val voltageThreshold: Float,

    val anchorQ: Int,
    val anchorR: Int,
    val anchorS: Int
) {
    companion object {
        fun fromTile(tile: CronTile): CronTileEntity {
            val coord = tile.anchorCoord
            return CronTileEntity(
```

```
                id = "${coord.q},${coord.r},${coord.s}",
                intervalFib = tile.intervalFib,
                lastTick = tile.lastTick,
                voltageThreshold = tile.voltageThreshold,
                anchorQ = coord.q,
                anchorR = coord.r,
                anchorS = coord.s
            )
        }
    }

    fun toTile(): CronTile = CronTile(
        intervalFib = intervalFib,
        lastTick = lastTick,
        voltageThreshold = voltageThreshold,
        anchorCoord = AxialCoord(anchorQ, anchorR, anchorS)
    )
}

/**
 * Room Entity for lattice metadata
 */
@Entity(tableName = "lattice_meta")
data class LatticeMetaEntity(
    @PrimaryKey
    val id: Int = 1, // Singleton

    val version: Int,
    val explorerVoltage: Float,
    val junctionThreshold: Byte,
    val radialRadius: Short,
    val cronFibBase: Byte,
    val lastUpdated: Long
)

@Dao
interface HexTileDao {
    @Query("SELECT * FROM hex_tiles ORDER BY evictionPriority DESC, priorityTiebreaker DESC")
    suspend fun getAllOrdered(): List

    @Query("SELECT * FROM hex_tiles WHERE q = :q AND r = :r AND s = :s")
    suspend fun getByCoord(q: Int, r: Int, s: Int): HexTileEntity?

    @Query("SELECT * FROM hex_tiles WHERE evictionPriority >= :minPriority")
    suspend fun getByMinPriority(minPriority: Byte): List

    @Query("SELECT COUNT(*) FROM hex_tiles")
    suspend fun count(): Int

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(tile: HexTileEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(tiles: List)

    @Delete
    suspend fun delete(tile: HexTileEntity)

    @Query("DELETE FROM hex_tiles WHERE id = :id")
    suspend fun deleteById(id: String)

    @Query("DELETE FROM hex_tiles ORDER BY evictionPriority DESC, priorityTiebreaker DESC LIMIT :count")
    suspend fun deleteOutermost(count: Int)

    @Query("DELETE FROM hex_tiles")
    suspend fun deleteAll()
}

@Dao
interface CronTileDao {
    @Query("SELECT * FROM cron_tiles")
    suspend fun getAll(): List

    @Query("SELECT * FROM cron_tiles WHERE anchorQ = :q AND anchorR = :r AND anchorS = :s")
    suspend fun getByAnchor(q: Int, r: Int, s: Int): CronTileEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(tile: CronTileEntity)

    @Delete
    suspend fun delete(tile: CronTileEntity)

    @Query("DELETE FROM cron_tiles WHERE id = :id")
    suspend fun deleteById(id: String)
}
```

```
@Dao
interface LatticeMetaDao {
    @Query("SELECT * FROM lattice_meta WHERE id = 1")
    suspend fun get(): LatticeMetaEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun set(meta: LatticeMetaEntity)
}


@Database(
    entities = [HexTileEntity::class, CronTileEntity::class, LatticeMetaEntity::class],
    version = 1,
    exportSchema = false
)
abstract class LatticeDatabase : RoomDatabase() {
    abstract fun hexTileDao(): HexTileDao
    abstract fun cronTileDao(): CronTileDao
    abstract fun latticeMetaDao(): LatticeMetaDao
}
```

data/LatticeRepository.kt (214 lines)

```
package com.glyf.cathedral.data

import android.content.Context
import androidx.room.Room
import com.glyf.cathedral.core.*
import kotlinx.coroutines.flow.*


/**
 * Repository for lattice operations
 * Bridges core models with Room persistence
 */
class LatticeRepository(context: Context) {

    private val db = Room.databaseBuilder(
        context,
        LatticeDatabase::class.java,
        "glyf_lattice"
    ).build()

    private val hexDao = db.hexTileDao()
    private val cronDao = db.cronTileDao()
    private val metaDao = db.latticeMetaDao()

    // ═══════════════════════════════════════════════════════════════════
    // HexTile Operations
    // ═══════════════════════════════════════════════════════════════════

    suspend fun getAllTiles(): List {
        return hexDao.getAllOrdered().map { it.toTile() }
    }

    suspend fun getTile(coord: AxialCoord): HexTile? {
        return hexDao.getByCoord(coord.q, coord.r, coord.s)?.toTile()
    }

    suspend fun insertTile(tile: HexTile) {
        hexDao.insert(HexTileEntity.fromTile(tile))
    }

    suspend fun insertTiles(tiles: List) {
        hexDao.insertAll(tiles.map { HexTileEntity.fromTile(it) })
    }

    suspend fun deleteTile(coord: AxialCoord) {
        hexDao.deleteById("${coord.q},${coord.r},${coord.s}")
    }

    suspend fun evictOutermost(count: Int) {
        hexDao.deleteOutermost(count)
    }

    suspend fun getTileCount(): Int = hexDao.count()

    fun getTilesFlow(): Flow> = flow {
```

```
        while (true) {
            emit(getAllTiles())
            kotlinx.coroutines.delay(1000) // Refresh every second
        }
    }

    // ═══════════════════════════════════════════════════════════════════════
    // CronTile Operations
    // ═══════════════════════════════════════════════════════════════════════

    suspend fun getAllCrons(): List {
        return cronDao.getAll().map { it.toTile() }
    }

    suspend fun getCron(anchor: AxialCoord): CronTile? {
        return cronDao.getByAnchor(anchor.q, anchor.r, anchor.s)?.toTile()
    }

    suspend fun insertCron(cron: CronTile) {
        cronDao.insert(CronTileEntity.fromTile(cron))
    }

    suspend fun deleteCron(anchor: AxialCoord) {
        cronDao.deleteById("${anchor.q},${anchor.r},${anchor.s}")
    }

    // ═══════════════════════════════════════════════════════════════════════
    // Lattice State Operations
    // ═══════════════════════════════════════════════════════════════════════

    suspend fun getLatticeState(): LatticeState {
        val meta = metaDao.get()
        val tiles = getAllTiles()
        val crons = getAllCrons()

        return if (meta != null) {
            LatticeState(
                version = meta.version,
                hotTiles = tiles,
                cronTiles = crons,
                explorerVoltage = meta.explorerVoltage,
                junctionThreshold = meta.junctionThreshold,
                radialRadius = meta.radialRadius,
                cronFibBase = meta.cronFibBase
            )
        } else {
            LatticeState() // Default
        }
    }

    suspend fun saveLatticeState(state: LatticeState) {
        // Save metadata
        metaDao.set(
            LatticeMetaEntity(
                version = state.version,
                explorerVoltage = state.explorerVoltage,
                junctionThreshold = state.junctionThreshold,
                radialRadius = state.radialRadius,
                cronFibBase = state.cronFibBase,
                lastUpdated = System.currentTimeMillis()
            )
        )

        // Note: Tiles and crons are saved individually to avoid full wipe
    }

    // ═══════════════════════════════════════════════════════════════════════
    // High-Level Operations
    // ═══════════════════════════════════════════════════════════════════════

    /**
     * Insert tile with auto-eviction if at capacity
     */
    suspend fun insertWithEviction(tile: HexTile, maxTiles: Int = 1024): HexTile? {
        val currentCount = getTileCount()

        return if (currentCount >= maxTiles) {
            // Evict outermost tile
            val allTiles = getAllTiles()
```

```
            if (allTiles.isNotEmpty()) {
                val toEvict = allTiles.maxByOrNull { it.evictionPriority.toInt() }
                toEvict?.let {
                    deleteTile(it.coord)
                    insertTile(tile)
                    it // Return evicted tile
                }
            } else {
                insertTile(tile)
                null
            }
        } else {
            insertTile(tile)
            null
        }
    }

    /**
     * Query tiles within radial distance
     */
    suspend fun queryRadial(center: AxialCoord, radius: Int): List {
        val allTiles = getAllTiles()
        return allTiles.filter { it.coord.distanceTo(center) <= radius }
    }

    /**
     * Process all cron triggers
     */
    suspend fun processCrons(nowSecs: Long = System.currentTimeMillis() / 1000): Int {
        val state = getLatticeState()
        val (newState, fired) = state.processCrons(nowSecs)

        // Save updated crons
        newState.cronTiles.forEach { insertCron(it) }

        return fired
    }

    /**
     * Clear all data
     */
    suspend fun clearAll() {
        hexDao.deleteAll()
        // Crons and meta are preserved? Or clear them too?
        // For now, keep crons as they're scheduled tasks
    }
}
```

UI Layer

`ui/visualization/HexLatticeView.kt` (329 lines)

```
package com.glyf.cathedral.ui.visualization

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.*
import androidx.compose.ui.graphics.drawscope.DrawScope
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.glyf.cathedral.core.*
import kotlin.math.*

/**
 * Hexagonal lattice visualization using Compose Canvas
 * Renders the φ⁷ lattice with golden-ratio scaling
 */
@Composable
fun HexLatticeView(
    tiles: List,
    selectedCoord: AxialCoord? = null,
```

```
    onTileSelected: (AxialCoord) -> Unit = {},
    onEmptySpaceSelected: (Offset) -> Unit = {},
    modifier: Modifier = Modifier,
    hexSize: Float = 50f,
    showVesica: Boolean = false,
    goldenSpiral: Boolean = false
) {
    val hexRadius = hexSize
    val hexHeight = hexRadius * 2
    val hexWidth = sqrt(3f) * hexRadius

    Canvas(
        modifier = modifier
            .fillMaxSize()
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    // Convert pixel to axial coordinates
                    val coord = pixelToAxial(offset, hexSize, centerX = size.width / 2f, centerY = size.height / 2f)
                    val tile = tiles.find { it.coord == coord }
                    if (tile != null) {
                        onTileSelected(coord)
                    } else {
                        onEmptySpaceSelected(offset)
                    }
                }
            }
    ) {
        val centerX = size.width / 2
        val centerY = size.height / 2

        // Draw golden spiral background if enabled
        if (goldenSpiral) {
            drawGoldenSpiral(centerX, centerY, hexSize)
        }

        // Draw vesica piscis overlaps if enabled
        if (showVesica) {
            drawVesicaOverlaps(tiles, centerX, centerY, hexSize)
        }

        // Draw all tiles
        tiles.forEach { tile ->
            val isSelected = tile.coord == selectedCoord
            drawHexTile(tile, centerX, centerY, hexSize, isSelected)
        }

        // Draw coordinate labels for selected
        selectedCoord?.let { coord ->
            val (px, py) = axialToPixel(coord, hexSize, centerX, centerY)
            drawContext.canvas.nativeCanvas.apply {
                val paint = android.graphics.Paint().apply {
                    color = android.graphics.Color.WHITE
                    textSize = 30f
                    textAlign = android.graphics.Paint.Align.CENTER
                }
                drawText("${coord.q},${coord.r},${coord.s}", px, py - hexSize - 20, paint)
            }
        }
    }
}

/**
 * Draw a single hex tile
 */
private fun DrawScope.drawHexTile(
    tile: HexTile,
    centerX: Float,
    centerY: Float,
    hexSize: Float,
    isSelected: Boolean
) {
    val (px, py) = axialToPixel(tile.coord, hexSize, centerX, centerY)
    val corners = getHexCorners(px, py, hexSize)

    // Color based on ternary spin and phi magnitude
    val baseColor = when (tile.spin) {
        TernarySpin.NEGATIVE -> Color(0xFFE57373) // Red
        TernarySpin.ZERO -> Color(0xFF64B5F6)    // Blue
        TernarySpin.POSITIVE -> Color(0xFF81C784) // Green
    }

    // Modulate alpha by phiMag / PHI_7
    val phiRatio = (tile.phiMag / PhiConstants.PHI_7.toFloat()).coerceIn(0f, 1f)
    val fillColor = baseColor.copy(alpha = 0.3f + phiRatio * 0.5f)

    // Draw hexagon
```

```
    drawPath(
        path = Path().apply {
            corners.forEachIndexed { index, offset ->
                if (index == 0) moveTo(offset.x, offset.y)
                else lineTo(offset.x, offset.y)
            }
            close()
        },
        color = fillColor
    )

    // Draw border
    drawPath(
        path = Path().apply {
            corners.forEachIndexed { index, offset ->
                if (index == 0) moveTo(offset.x, offset.y)
                else lineTo(offset.x, offset.y)
            }
            close()
        },
        color = if (isSelected) Color.White else baseColor,
        style = Stroke(width = if (isSelected) 4f else 2f)
    )

    // Draw chiral hash indicator (small inner circle)
    val chiralIntensity = ((tile.chiralHash and 0xFF) / 255.0).toFloat()
    drawCircle(
        color = Color.White.copy(alpha = chiralIntensity),
        radius = hexSize * 0.2f,
        center = Offset(px, py)
    )

    // Draw priority indicator (outer ring thickness)
    val priorityRatio = tile.evictionPriority.toInt() / 255f
    if (priorityRatio > 0.5f) {
        drawCircle(
            color = Color.Yellow.copy(alpha = priorityRatio * 0.5f),
            radius = hexSize * 0.9f,
            center = Offset(px, py),
            style = Stroke(width = 3f)
        )
    }
}

/**
 * Draw golden spiral background
 */
private fun DrawScope.drawGoldenSpiral(centerX: Float, centerY: Float, hexSize: Float) {
    val phi = PhiConstants.PHI.toFloat()
    val goldenAngle = PhiConstants.GOLDEN_ANGLE_RAD.toFloat()

    var radius = hexSize
    var angle = 0f

    repeat(100) { i ->
        val x = centerX + radius * cos(angle)
        val y = centerY + radius * sin(angle)

        val alpha = (1f - i / 100f) * 0.3f
        drawCircle(
            color = Color(0xFFFFD700).copy(alpha = alpha),
            radius = 4f,
            center = Offset(x, y)
        )

        radius *= 1.02f
        angle += goldenAngle
    }
}

/**
 * Draw vesica piscis overlaps between nearby tiles
 */
private fun DrawScope.drawVesicaOverlaps(
    tiles: List,
    centerX: Float,
    centerY: Float,
    hexSize: Float
) {
    // Find overlapping pairs (distance == 1)
    val pairs = mutableListOf>()

    tiles.forEachIndexed { i, tile1 ->
        tiles.drop(i + 1).forEach { tile2 ->
            if (tile1.coord.distanceTo(tile2.coord) == 1) {
                pairs.add(tile1 to tile2)
```

```
            }
        }
    }

    // Draw vesica piscis for each pair
    pairs.forEach { (t1, t2) ->
        val (x1, y1) = axialToPixel(t1.coord, hexSize, centerX, centerY)
        val (x2, y2) = axialToPixel(t2.coord, hexSize, centerX, centerY)

        val midX = (x1 + x2) / 2
        val midY = (y1 + y2) / 2
        val distance = sqrt((x2 - x1).pow(2) + (y2 - y1).pow(2))

        // Vesica lens (simplified as ellipse)
        drawOval(
            color = Color(0xFF9C27B0).copy(alpha = 0.2f),
            topLeft = Offset(midX - distance/4, midY - distance/6),
            size = androidx.compose.ui.geometry.Size(distance/2, distance/3)
        )
    }
}

/**
 * Convert axial coordinates to pixel position
 */
private fun axialToPixel(
    coord: AxialCoord,
    hexSize: Float,
    centerX: Float,
    centerY: Float
): Pair {
    val q = coord.q.toFloat()
    val r = coord.r.toFloat()

    val x = hexSize  (sqrt(3f)  q + sqrt(3f) / 2 * r)
    val y = hexSize  (3f / 2  r)

    return Pair(centerX + x, centerY + y)
}

/**
 * Convert pixel position to axial coordinates (approximate)
 */
private fun pixelToAxial(
    offset: Offset,
    hexSize: Float,
    centerX: Float,
    centerY: Float
): AxialCoord {
    val x = (offset.x - centerX) / hexSize
    val y = (offset.y - centerY) / hexSize

    val q = (sqrt(3f) / 3  x - 1f / 3  y)
    val r = (2f / 3 * y)
    val s = -q - r

    return cubeRound(q, r, s)
}

/**
 * Round floating point cube coordinates to nearest hex
 */
private fun cubeRound(q: Float, r: Float, s: Float): AxialCoord {
    var rq = q.roundToInt()
    var rr = r.roundToInt()
    var rs = s.roundToInt()

    val dq = abs(rq - q)
    val dr = abs(rr - r)
    val ds = abs(rs - s)

    if (dq > dr && dq > ds) {
        rq = -rr - rs
    } else if (dr > ds) {
        rr = -rq - rs
    } else {
        rs = -rq - rr
    }

    return AxialCoord(rq, rr, rs)
}


/**
 * Get the 6 corners of a hexagon
 */
```

```
private fun getHexCorners(centerX: Float, centerY: Float, size: Float): List {
    return (0 until 6).map { i ->
        val angle = (Math.PI / 3 * i - Math.PI / 6).toFloat()
        Offset(
            centerX + size * cos(angle),
            centerY + size * sin(angle)
        )
    }
}
```

`ui/LatticeScreens.kt` (233 lines)

```
package com.glyf.cathedral.ui

import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.glyf.cathedral.core.*
import com.glyf.cathedral.ui.visualization.HexLatticeView

/**
 * Main lattice screen with controls
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LatticeScreen(
    tiles: List,
    explorerVoltage: Float,
    onTileSelected: (AxialCoord) -> Unit,
    onAddTile: (AxialCoord) -> Unit,
    onEvictOutermost: () -> Unit,
    onScaleVoltage: (Int) -> Unit,
    onProcessCrons: () -> Unit,
    modifier: Modifier = Modifier
) {
    var selectedCoord by remember { mutableStateOf(null) }
    var showVesica by remember { mutableStateOf(false) }
    var showSpiral by remember { mutableStateOf(true) }
    var hexSize by remember { mutableStateOf(50f) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("GLYF Cathedral φ⁷") },
                actions = {
                    IconButton(onClick = { showVesica = !showVesica }) {
                        Icon(
                            imageVector = if (showVesica) Icons.Default.Visibility else Icons.Default.VisibilityOff,
                            contentDescription = "Toggle Vesica"
                        )
                    }
                    IconButton(onClick = { showSpiral = !showSpiral }) {
                        Icon(
                            imageVector = if (showSpiral) Icons.Default.Refresh else Icons.Default.Add,
                            contentDescription = "Toggle Spiral"
                        )
                    }
                }
            )
        },
        floatingActionButton = {
            Column {
                SmallFloatingActionButton(
                    onClick = { hexSize = (hexSize * 1.2f).coerceAtMost(100f) }
                ) {
                    Icon(Icons.Default.Add, "Zoom In")
                }
                Spacer(modifier = Modifier.height(8.dp))
                SmallFloatingActionButton(
                    onClick = { hexSize = (hexSize / 1.2f).coerceAtLeast(20f) }
                ) {
                    Icon(Icons.Default.Remove, "Zoom Out")
                }
                Spacer(modifier = Modifier.height(8.dp))
                FloatingActionButton(onClick = onEvictOutermost) {
                    Icon(Icons.Default.Delete, "Evict Outermost")
                }
            }
        }
```

```
    ) { padding ->
        Column(
            modifier = modifier
                .fillMaxSize()
                .padding(padding)
        ) {
            // Status bar
            StatusBar(
                tileCount = tiles.size,
                explorerVoltage = explorerVoltage,
                selectedCoord = selectedCoord,
                modifier = Modifier.fillMaxWidth()
            )

            // Hex lattice visualization
            HexLatticeView(
                tiles = tiles,
                selectedCoord = selectedCoord,
                onTileSelected = { coord ->
                    selectedCoord = coord
                    onTileSelected(coord)
                },
                onEmptySpaceSelected = { offset ->
                    // Convert offset to coord and add tile
                    // Simplified for demo
                },
                modifier = Modifier.weight(1f),
                hexSize = hexSize,
                showVesica = showVesica,
                goldenSpiral = showSpiral
            )

            // Control panel
            ControlPanel(
                onScaleUp = { onScaleVoltage(1) },
                onScaleDown = { onScaleVoltage(-1) },
                onProcessCrons = onProcessCrons,
                modifier = Modifier.fillMaxWidth()
            )
        }
    }
}

@Composable
private fun StatusBar(
    tileCount: Int,
    explorerVoltage: Float,
    selectedCoord: AxialCoord?,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier,
        tonalElevation = 2.dp
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Column {
                Text("Tiles: $tileCount / 1024", style = MaterialTheme.typography.bodyMedium)
                LinearProgressIndicator(
                    progress = tileCount / 1024f,
                    modifier = Modifier.width(100.dp)
                )
            }

            Column(horizontalAlignment = androidx.compose.ui.Alignment.CenterHorizontally) {
                Text("φ Voltage", style = MaterialTheme.typography.labelSmall)
                Text(
                    "%.2f".format(explorerVoltage),
                    style = MaterialTheme.typography.titleMedium
                )
            }

            Column(horizontalAlignment = androidx.compose.ui.Alignment.End) {
                Text("Selected:", style = MaterialTheme.typography.labelSmall)
                Text(
                    selectedCoord?.let { "${it.q},${it.r},${it.s}" } ?: "None",
                    style = MaterialTheme.typography.bodyMedium
                )
            }
        }
    }
}
```

```
@Composable
private fun ControlPanel(
    onScaleUp: () -> Unit,
    onScaleDown: () -> Unit,
    onProcessCrons: () -> Unit,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier,
        tonalElevation = 4.dp
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement = Arrangement.SpaceEvenly
        ) {
            Button(onClick = onScaleUp) {
                Icon(Icons.Default.KeyboardArrowUp, null)
                Spacer(Modifier.width(4.dp))
                Text("Scale φ")
            }

            Button(onClick = onScaleDown) {
                Icon(Icons.Default.KeyboardArrowDown, null)
                Spacer(Modifier.width(4.dp))
                Text("Scale φ⁻¹")
            }

            Button(onClick = onProcessCrons) {
                Icon(Icons.Default.Notifications, null)
                Spacer(Modifier.width(4.dp))
                Text("Process Crons")
            }
        }
    }
}


 /**
 * Settings screen
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SettingsScreen(
    radialRadius: Int,
    onRadialRadiusChange: (Int) -> Unit,
    cronEnabled: Boolean,
    onCronEnabledChange: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Settings") })
        }
    ) { padding ->
        Column(
            modifier = modifier
                .fillMaxSize()
                .padding(padding)
                .padding(16.dp)
        ) {
            // Radial radius setting
            ListItem(
                headlineContent = { Text("Radial Query Radius") },
                supportingContent = { Text("Maximum distance for lattice queries") },
                trailingContent = {
                    Text("$radialRadius", style = MaterialTheme.typography.titleMedium)
                }
            )

            Slider(
                value = radialRadius.toFloat(),
                onValueChange = { onRadialRadiusChange(it.toInt()) },
                valueRange = 5f..50f,
                steps = 44
            )

            Divider(modifier = Modifier.padding(vertical = 16.dp))

            // Cron toggle
            ListItem(
```

```
                headlineContent = { Text("Fibonacci Cron") },
                supportingContent = { Text("Periodic lattice persistence (13-min intervals)") },
                trailingContent = {
                    Switch(
                        checked = cronEnabled,
                        onCheckedChange = onCronEnabledChange
                    )
                }
            )

            Divider(modifier = Modifier.padding(vertical = 16.dp))

            // About
            ListItem(
                headlineContent = { Text("GLYF Cathedral") },
                supportingContent = { Text("Version 0.7.2 φ⁷") }
            )
        }
    }
}
```

ui/LatticeViewModel.kt (180 lines)

```
package com.glyf.cathedral.ui

import android.app.Application
import androidx.lifecycle.*
import com.glyf.cathedral.core.*
import com.glyf.cathedral.cron.LatticeCronWorker
import com.glyf.cathedral.data.LatticeRepository
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.launch

/**
 * ViewModel for lattice operations
 * Bridges UI with repository and cron workers
 */
class LatticeViewModel(application: Application) : AndroidViewModel(application) {

    private val repository = LatticeRepository(application)

    // ═══════════════════════════════════════════════════════════════════
    // State Flows
    // ═══════════════════════════════════════════════════════════════════

    private val _tiles = MutableStateFlow>(emptyList())
    val tiles: StateFlow> = _tiles.asStateFlow()

    private val _explorerVoltage = MutableStateFlow(PhiConstants.PHI_7.toFloat())
    val explorerVoltage: StateFlow = _explorerVoltage.asStateFlow()

    private val _selectedCoord = MutableStateFlow(null)
    val selectedCoord: StateFlow = _selectedCoord.asStateFlow()

    private val _radialRadius = MutableStateFlow(21)
    val radialRadius: StateFlow = _radialRadius.asStateFlow()

    private val _cronEnabled = MutableStateFlow(false)
    val cronEnabled: StateFlow = _cronEnabled.asStateFlow()

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow = _isLoading.asStateFlow()

    // ═══════════════════════════════════════════════════════════════════
    // Initialization
    // ═══════════════════════════════════════════════════════════════════

    init {
        viewModelScope.launch {
            loadLattice()
        }

        // Auto-refresh tiles every second
        viewModelScope.launch {
            while (true) {
 kotlinx.coroutines.delay(1000)
                if (!_isLoading.value) {
                    refreshTiles()
                }
            }
        }
    }
```

```
    // ═══════════════════════════════════════════════════════════════════════
    // Public Operations
    // ═══════════════════════════════════════════════════════════════════════

    fun selectCoord(coord: AxialCoord) {
        _selectedCoord.value = coord
    }

    fun addTileAt(coord: AxialCoord, spin: TernarySpin = TernarySpin.ZERO) {
        viewModelScope.launch {
            _isLoading.value = true

            val tile = HexTile(
                coord = coord,
                spin = spin,
                phiMag = _explorerVoltage.value * 0.5f
            ).withPriority(_radialRadius.value)

            val evicted = repository.insertWithEviction(tile)
            evicted?.let {
                // Handle evicted tile (log, sync, etc.)
            }

            refreshTiles()
            _isLoading.value = false
        }
    }

    fun evictOutermost() {
        viewModelScope.launch {
            _isLoading.value = true
            repository.evictOutermost(1)
            refreshTiles()
            _isLoading.value = false
        }
    }

    fun scaleVoltage(k: Int) {
        viewModelScope.launch {
            _isLoading.value = true

            val currentState = repository.getLatticeState()
            val newState = currentState.scaleVoltage(k)
            repository.saveLatticeState(newState)

            _explorerVoltage.value = newState.explorerVoltage
            refreshTiles()
            _isLoading.value = false
        }
    }

    fun processCrons() {
        viewModelScope.launch {
            _isLoading.value = true
            val fired = repository.processCrons()
            if (fired > 0) {
                // Show notification or update UI
            }
            refreshTiles()
            _isLoading.value = false
        }
    }

    fun updateRadialRadius(radius: Int) {
        _radialRadius.value = radius
        viewModelScope.launch {
            // Update all tile priorities
            val state = repository.getLatticeState()
            val updatedTiles = state.hotTiles.map {
                it.withPriority(radius)
            }

            // Clear and re-insert with new priorities
            repository.clearAll()
            updatedTiles.forEach { repository.insertTile(it) }

            refreshTiles()
        }
    }

    fun setCronEnabled(enabled: Boolean) {
        _cronEnabled.value = enabled

        if (enabled) {
            LatticeCronWorker.schedule(getApplication())
```

```
        } else {
            LatticeCronWorker.cancel(getApplication())
        }
    }

    fun queryRadial(center: AxialCoord, radius: Int = _radialRadius.value) {
        viewModelScope.launch {
            _isLoading.value = true
            val results = repository.queryRadial(center, radius)
            // Could emit these to a separate flow for display
            _isLoading.value = false
        }
    }

    // ═══════════════════════════════════════════════════════════════════
    // Private Helpers
    // ═══════════════════════════════════════════════════════════════════

    private suspend fun loadLattice() {
        _isLoading.value = true

        val state = repository.getLatticeState()
        _explorerVoltage.value = state.explorerVoltage
        _radialRadius.value = state.radialRadius.toInt()

        refreshTiles()
        _isLoading.value = false
    }

    private suspend fun refreshTiles() {
        val tiles = repository.getAllTiles()
        _tiles.value = tiles
    }
}


/**
 * Factory for creating ViewModel with Application context
 */
class LatticeViewModelFactory(
    private val application: Application
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun  create(modelClass: Class): T {
        if (modelClass.isAssignableFrom(LatticeViewModel::class.java)) {
            return LatticeViewModel(application) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

Cron System

`cron/LatticeWorkers.kt` (246 lines)

```
package com.glyf.cathedral.cron

import android.content.Context
import androidx.work.*
import com.glyf.cathedral.core.PhiConstants
import com.glyf.cathedral.data.LatticeRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.util.concurrent.TimeUnit

/**
 * WorkManager cron worker for φ⁷ lattice persistence
 * Fibonacci-timed execution with voltage threshold checks
 */
class LatticeCronWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    companion object {
        const val WORK_NAME = "glyf_lattice_cron"
        const val KEY_INTERVAL_INDEX = "interval_index"
        const val KEY_FIRE_COUNT = "fire_count"

        /**
```

```
         * Schedule the cron worker with Fibonacci intervals
         */
        fun schedule(context: Context) {
            val constraints = Constraints.Builder()
                .setRequiresBatteryNotLow(true)
                .build()

            // Initial trigger after 13 minutes (default Fibonacci interval)
            val initialDelay = PhiConstants.FIB_MOD_89[13] * 60L

            val workRequest = PeriodicWorkRequestBuilder(
                initialDelay, TimeUnit.SECONDS
            )
                .setConstraints(constraints)
                .setBackoffCriteria(
                    BackoffPolicy.EXPONENTIAL,
                    WorkRequest.MIN_BACKOFF_MILLIS,
                    TimeUnit.MILLISECONDS
                )
                .addTag(WORK_NAME)
                .build()

            WorkManager.getInstance(context).enqueueUniquePeriodicWork(
                WORK_NAME,
                ExistingPeriodicWorkPolicy.KEEP,
                workRequest
            )
        }

        /**
         * Cancel scheduled work
         */
        fun cancel(context: Context) {
            WorkManager.getInstance(context).cancelUniqueWork(WORK_NAME)
        }
    }

    override suspend fun doWork(): Result = withContext(Dispatchers.IO) {
        try {
            val repository = LatticeRepository(applicationContext)
            val fired = repository.processCrons()

            if (fired > 0) {
                // Trigger CBOR emission if crons fired
                // TODO: Integrate with Rosetta bridge
                emitRosettaInquiry()
            }

            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }

    private fun emitRosettaInquiry() {
        // Placeholder for CBOR emission
        // Will be implemented when Rosetta bridge is ready
    }
}

/**
 * One-time worker for immediate lattice operations
 */
class LatticeImmediateWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    companion object {
        const val KEY_OPERATION = "operation"
        const val OP_EVICT = "evict"
        const val OP_SCALE_VOLTAGE = "scale_voltage"
        const val KEY_PARAM = "param"

        fun enqueueEvict(context: Context, count: Int) {
            val workRequest = OneTimeWorkRequestBuilder()
                .setInputData(workDataOf(
                    KEY_OPERATION to OP_EVICT,
                    KEY_PARAM to count
                ))
                .build()

            WorkManager.getInstance(context).enqueue(workRequest)
        }

        fun enqueueScaleVoltage(context: Context, k: Int) {
```

```
            val workRequest = OneTimeWorkRequestBuilder()
                .setInputData(workDataOf(
                    KEY_OPERATION to OP_SCALE_VOLTAGE,
                    KEY_PARAM to k
                ))
                .build()

            WorkManager.getInstance(context).enqueue(workRequest)
        }
    }

    override suspend fun doWork(): Result = withContext(Dispatchers.IO) {
        val repository = LatticeRepository(applicationContext)
        val operation = inputData.getString(KEY_OPERATION)

        when (operation) {
            OP_EVICT -> {
                val count = inputData.getInt(KEY_PARAM, 1)
                repository.evictOutermost(count)
            }
            OP_SCALE_VOLTAGE -> {
                val k = inputData.getInt(KEY_PARAM, 0)
                val state = repository.getLatticeState()
                val newState = state.scaleVoltage(k)
                repository.saveLatticeState(newState)
            }
        }

        Result.success()
    }
}


/**
 * Worker for 72-hour stress test simulation
 */
class LatticeStressTestWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    companion object {
        const val WORK_NAME = "glyf_stress_test"

        fun start(context: Context) {
            val workRequest = OneTimeWorkRequestBuilder()
                .setInitialDelay(0, TimeUnit.SECONDS)
                .build()

            WorkManager.getInstance(context).enqueueUniqueWork(
                WORK_NAME,
                ExistingWorkPolicy.REPLACE,
                workRequest
            )
        }
    }

    override suspend fun doWork(): Result = withContext(Dispatchers.IO) {
        val repository = LatticeRepository(applicationContext)
        val random = java.util.Random()

        // Run for 72 hours of simulation (or until stopped)
        val iterations = 72  60  60 // 72 hours in seconds

        repeat(iterations) { i ->
            // Random mutation: insert or evict
            if (random.nextBoolean()) {
                val q = random.nextInt(20) - 10
                val r = random.nextInt(20) - 10
                val tile = HexTile(
                    coord = AxialCoord.fromQr(q, r),
                    spin = TernarySpin.values()[random.nextInt(3)],
                    phiMag = random.nextFloat() * PhiConstants.PHI_7.toFloat()
                )
                repository.insertWithEviction(tile)
            } else {
                repository.evictOutermost(1)
            }

            // Every 13 iterations, process crons
            if (i % 13 == 0) {
                repository.processCrons()
```

```
            }

            // Report progress every hour
            if (i % 3600 == 0) {
                val hours = i / 3600
                setProgress(workDataOf("hours" to hours))
            }

            // Small delay to not overwhelm the system
            kotlinx.coroutines.delay(1000) // 1 second per iteration
        }

        Result.success()
    }
}
```

`cron/CronInfrastructure.kt` (50 lines)

```
package com.glyf.cathedral.cron

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import androidx.work.*

/**
 * Boot receiver to reschedule cron work after reboot
 */
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_BOOT_COMPLETED) {
            // Reschedule cron work
            LatticeCronWorker.schedule(context)
        }
    }
}


/**
 * Custom worker factory for dependency injection
 */
class LatticeWorkerFactory : WorkerFactory() {
    override fun createWorker(
        context: Context,
        workerClassName: String,
        workerParameters: WorkerParameters
    ): ListenableWorker? {
        return when (workerClassName) {
            LatticeCronWorker::class.java.name ->
                LatticeCronWorker(context, workerParameters)
            LatticeImmediateWorker::class.java.name ->
                LatticeImmediateWorker(context, workerParameters)
            LatticeStressTestWorker::class.java.name ->
                LatticeStressTestWorker(context, workerParameters)
            else -> null
        }
    }
}
```

Rosetta Bridge

`rosetta/RosettaTelegramBridge.kt` (234 lines)

```
package com.glyf.cathedral.rosetta

import android.content.Context
import io.ktor.client.*
import io.ktor.client.call.*
import io.ktor.client.engine.android.*
import io.ktor.client.plugins.*
import io.ktor.client.request.*
import io.ktor.http.*
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
```

```
/**
 * Rosetta Protocol Telegram Bridge
 * Implements CBOR-encoded Inquiry Vectors via Telegram Bot API
 */
class RosettaTelegramBridge(
    private val context: Context,
    private val botToken: String
) {
    private val client = HttpClient(Android) {
        install(HttpTimeout) {
            requestTimeoutMillis = 30000
            connectTimeoutMillis = 10000
        }
    }

    private val json = Json { ignoreUnknownKeys = true }

    companion object {
        const val CBOR_TAG = 0xD9F7
        const val INQUIRY_VECTOR_SIZE = 32 // bytes
    }

    /**
     * Send Inquiry Vector as inline query result
     * Encodes lattice state as CBOR + base64url for Telegram
     */
    suspend fun emitInquiryVector(
        chatId: Long,
        latticeState: ByteArray // CBOR-encoded lattice snapshot
    ): Result {
        return try {
            // Encode to base64url
            val encoded = base64UrlEncode(latticeState)

            // Send as inline query result article
            val response = client.post("https://api.telegram.org/bot$botToken/sendMessage") {
                contentType(ContentType.Application.Json)
                setBody(Json.encodeToString(
                    SendMessageRequest(
                        chatId = chatId,
                        text = "▫ GLYF Inquiry Vector\n$encoded",
                        parseMode = "MarkdownV2"
                    )
                ))
            }

            val result = response.body()
            if (result.ok) {
                Result.success(result.result!!)
            } else {
                Result.failure(Exception(result.description ?: "Unknown error"))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    /**
     * Decode incoming Inquiry Vector
     * base64url → CBOR → lattice state
     */
    fun decodeInquiryVector(encoded: String): Result {
        return try {
            val cborBytes = base64UrlDecode(encoded)
            // TODO: Validate CBOR tag 0xD9F7
            // TODO: Parse 32-byte inquiry vector structure
            Result.success(cborBytes)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    /**
     * Query remote lattice via φ⁷-radial search
     */
    suspend fun queryRemoteLattice(
        targetChatId: Long,
        centerQ: Int,
        centerR: Int,
        radius: Int
    ): Result> {
        // Build CBOR inquiry vector
        val inquiry = buildInquiryVector(centerQ, centerR, radius)

        return try {
            val response = client.post("https://api.telegram.org/bot$botToken/sendMessage") {
```

```
                contentType(ContentType.Application.Json)
                setBody(Json.encodeToString(
                    QueryRequest(
                        chatId = targetChatId,
                        inquiryVector = base64UrlEncode(inquiry)
                    )
                ))
            }

            // Parse response (would come via webhook or polling)
            Result.success(emptyList()) // Placeholder
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    // ═══════════════════════════════════════════════════════════════════════
    // Encoding Utilities
    // ═══════════════════════════════════════════════════════════════════════

    private fun base64UrlEncode(data: ByteArray): String {
        return java.util.Base64.getUrlEncoder()
            .withoutPadding()
            .encodeToString(data)
    }

    private fun base64UrlDecode(encoded: String): ByteArray {
        return java.util.Base64.getUrlDecoder()
            .decode(encoded)
    }

    /**
     * Build 32-byte CBOR Inquiry Vector
     * Format: [tag:2][qrs:6][radius:2][phi_mag:4][hash:16][reserved:2]
     */
    private fun buildInquiryVector(q: Int, r: Int, radius: Int): ByteArray {
        val bytes = ByteArray(INQUIRY_VECTOR_SIZE)

        // Tag (0xD9F7 = 55799 in CBOR major type 6)
        bytes[0] = 0xD9.toByte()
        bytes[1] = 0xF7.toByte()

        // Q, R coordinates (3 bytes each, big-endian signed)
        bytes[2] = (q shr 16).toByte()
        bytes[3] = (q shr 8).toByte()
        bytes[4] = q.toByte()
        bytes[5] = (r shr 16).toByte()
        bytes[6] = (r shr 8).toByte()
        bytes[7] = r.toByte()

        // Radius (2 bytes)
        bytes[8] = (radius shr 8).toByte()
        bytes[9] = radius.toByte()

        // Phi magnitude (4 bytes, IEEE 754 float)
        val phiBits = java.lang.Float.floatToIntBits(PhiConstants.PHI_7.toFloat())
        bytes[10] = (phiBits shr 24).toByte()
        bytes[11] = (phiBits shr 16).toByte()
        bytes[12] = (phiBits shr 8).toByte()
        bytes[13] = phiBits.toByte()

        // Chiral hash (16 bytes) - truncated
        // Would be computed from lattice state

        // Reserved (2 bytes)

        return bytes
    }

    fun close() {
        client.close()
    }
}

// ═══════════════════════════════════════════════════════════════════════
// Data Classes for Telegram API
// ═══════════════════════════════════════════════════════════════════════

@Serializable
data class SendMessageRequest(
    val chatId: Long,
    val text: String,
    val parseMode: String? = null
)

@Serializable
data class QueryRequest(
```

```
    val chatId: Long,
    val inquiryVector: String
)

@Serializable
data class TelegramResponse(
    val ok: Boolean,
    val result: Message? = null,
    val description: String? = null
)

@Serializable
data class Message(
    val messageId: Long,
    val chat: Chat,
    val text: String? = null,
    val date: Long
)

@Serializable
data class Chat(
    val id: Long,
    val type: String
)


/**
 * Remote tile response from lattice query
 */
data class RemoteTile(
    val q: Int,
    val r: Int,
    val s: Int,
    val spin: Byte,
    val phiMag: Float,
    val chiralHash: Long
)
```

Build Configuration

`MainActivity.kt` (123 lines)

```
package com.glyf.cathedral

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import com.glyf.cathedral.ui.*
import com.glyf.cathedral.ui.theme.GlyfCathedralTheme

/**
 * Main entry point for GLYF Cathedral Android
 */
class MainActivity : ComponentActivity() {

    private val viewModel: LatticeViewModel by viewModels {
        LatticeViewModelFactory(application)
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            GlyfCathedralTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    CathedralApp(viewModel)
                }
            }
        }
```

```
    }
}

/**
 * Main app with bottom navigation
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CathedralApp(viewModel: LatticeViewModel) {
    var selectedTab by remember { mutableStateOf(0) }

    val tiles by viewModel.tiles.collectAsState()
    val explorerVoltage by viewModel.explorerVoltage.collectAsState()
    val selectedCoord by viewModel.selectedCoord.collectAsState()
    val radialRadius by viewModel.radialRadius.collectAsState()
    val cronEnabled by viewModel.cronEnabled.collectAsState()

    Scaffold(
        bottomBar = {
            NavigationBar {
                NavigationBarItem(
                    icon = { Icon(Icons.Default.Home, contentDescription = "Lattice") },
                    label = { Text("Lattice") },
                    selected = selectedTab == 0,
                    onClick = { selectedTab = 0 }
                )
                NavigationBarItem(
                    icon = { Icon(Icons.Default.Settings, contentDescription = "Settings") },
                    label = { Text("Settings") },
                    selected = selectedTab == 1,
                    onClick = { selectedTab = 1 }
                )
            }
        }
    ) { padding ->
        when (selectedTab) {
            0 -> LatticeScreen(
                tiles = tiles,
                explorerVoltage = explorerVoltage,
                onTileSelected = { viewModel.selectCoord(it) },
                onAddTile = { coord ->
                    // Show dialog to select spin before adding
                    viewModel.addTileAt(coord)
                },
                onEvictOutermost = { viewModel.evictOutermost() },
                onScaleVoltage = { k -> viewModel.scaleVoltage(k) },
                onProcessCrons = { viewModel.processCrons() },
                modifier = Modifier.fillMaxSize()
            )
            1 -> SettingsScreen(
                radialRadius = radialRadius,
                onRadialRadiusChange = { viewModel.updateRadialRadius(it) },
                cronEnabled = cronEnabled,
                onCronEnabledChange = { viewModel.setCronEnabled(it) },
                modifier = Modifier.fillMaxSize()
            )
        }
    }
}

/**
 * Theme definition
 */
package com.glyf.cathedral.ui.theme

import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color

private val Purple80 = Color(0xFFD0BCFF)
private val PurpleGrey80 = Color(0xFFCCC2DC)
private val Pink80 = Color(0xFFEFB8C8)

private val Purple40 = Color(0xFF6650a4)
private val PurpleGrey40 = Color(0xFF625b71)
private val Pink40 = Color(0xFF7D5260)

private val GoldPhi = Color(0xFFFFD700)
private val DeepPurple = Color(0xFF4A148C)

@Composable
fun GlyfCathedralTheme(
    darkTheme: Boolean = true, // Default to dark for cathedral aesthetic
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) {
        darkColorScheme(
```

```
            primary = GoldPhi,
            secondary = Purple80,
            tertiary = Pink80,
            background = DeepPurple,
            surface = Color(0xFF1A0033),
            onPrimary = Color.Black,
            onBackground = Color.White
        )
    } else {
        lightColorScheme(
            primary = Purple40,
            secondary = PurpleGrey40,
            tertiary = Pink40
        )
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography(),
        content = content
    )
}
```

`CathedralApplication.kt` (16 lines)

```
package com.glyf.cathedral

import android.app.Application
import androidx.work.Configuration
import com.glyf.cathedral.cron.LatticeWorkerFactory

/**
 * Application class for GLYF Cathedral
 */
class CathedralApplication : Application(), Configuration.Provider {

    override fun getWorkManagerConfiguration(): Configuration {
        return Configuration.Builder()
            .setWorkerFactory(LatticeWorkerFactory())
            .build()
    }
}
```

`app/build.gradle.kts` (95 lines)

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.serialization")
    id("com.google.devtools.ksp")
}

android {
    namespace = "com.glyf.cathedral"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.glyf.cathedral"
        minSdk = 26
        targetSdk = 34
        versionCode = 702
        versionName = "0.7.2-phi7"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
        vectorDrawables {
            useSupportLibrary = true
        }
    }

    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
```

```
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = "17"
    }
    buildFeatures {
        compose = true
    }
    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.5"
    }
    packaging {
        resources {
            excludes += "/META-INF/{AL2.0,LGPL2.1}"
        }
    }
}

dependencies {
    // Core Android
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.6.2")
    implementation("androidx.activity:activity-compose:1.8.1")

    // Compose BOM
    val composeBom = platform("androidx.compose:compose-bom:2023.10.01")
    implementation(composeBom)
    androidTestImplementation(composeBom)

    // Compose UI
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-graphics")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")

    // Navigation
    implementation("androidx.navigation:navigation-compose:2.7.5")

    // ViewModel
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.2")

    // Room (Database)
    val roomVersion = "2.6.1"
    implementation("androidx.room:room-runtime:$roomVersion")
    implementation("androidx.room:room-ktx:$roomVersion")
    ksp("androidx.room:room-compiler:$roomVersion")

    // WorkManager (Cron)
    implementation("androidx.work:work-runtime-ktx:2.9.0")

    // Ktor (Networking for Telegram)
    val ktorVersion = "2.3.6"
    implementation("io.ktor:ktor-client-android:$ktorVersion")
    implementation("io.ktor:ktor-client-content-negotiation:$ktorVersion")
    implementation("io.ktor:ktor-serialization-kotlinx-json:$ktorVersion")

    // Serialization
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.2")

    // DataStore (Preferences)
    implementation("androidx.datastore:datastore-preferences:1.0.0")

    // Testing
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.3")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")

    // Debug
    debugImplementation("androidx.compose.ui:ui-tooling")
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

Summary

```
|------|-------|---------|
```

File	Lines	Purpose
`Models.kt`	349	Core φ^7 data structures
`Entities.kt`	192	Room database entities
`LatticeRepository.kt`	214	Data access layer
`HexLatticeView.kt`	329	Canvas visualization
`LatticeScreens.kt`	233	Compose UI screens
`LatticeViewModel.kt`	180	State management
`LatticeWorkers.kt`	246	Cron workers
`CronInfrastructure.kt`	50	Boot receiver
`RosettaTelegramBridge.kt`	234	Telegram protocol
`MainActivity.kt`	123	Entry point
`CathedralApplication.kt`	16	Application class
`app/build.gradle.kts`	95	Build config
Total	**2,281**	**Complete source**

--- GLYF Cathedral Android v0.7.2 φ^7 — Generated 2026-03-23